

Módulo 11 – Capítulo 10 – Sección 01

Diagnóstico inicial: evaluar el estado actual del stack, los datos y las capacidades del equipo

El diagnóstico inicial de una organización enterprise que quiere adoptar IA de manera sistemática no es un cuestionario de satisfacción ni un inventario de aspiraciones: es una auditoría técnica con evidencia verificable que produce un mapa del estado real de tres dimensiones simultáneamente. Un diagnóstico que tarda más de 6 semanas o que produce conclusiones vagas ("necesitamos mejorar nuestra madurez de datos") sin especificaciones accionables es una señal de que la organización no está lista para la velocidad que requiere la adopción de IA — o de que el proceso de diagnóstico fue diseñado para producir un informe, no para producir decisiones.

El **diagnóstico del stack tecnológico** es el más rápido de realizar: requiere un inventario de lo que existe y una comparación contra lo que requiere el caso de uso objetivo. Las preguntas clave son: ¿qué servicios cloud están disponibles y configurados? ¿Existe infraestructura de CI/CD (GitHub Actions, GitLab CI, Jenkins) o el despliegue de código es manual? ¿Qué plataformas de datos existen (data lake, data warehouse, bases de datos) y son accesibles programáticamente? ¿Hay observabilidad desplegada (Prometheus, Grafana, Datadog, CloudWatch) o el monitoreo es reactivo? Las respuestas a estas preguntas determinan qué componentes de la arquitectura de referencia del Capítulo 02 están disponibles y cuáles deben construirse o aprovisionarse desde cero.

El **diagnóstico de la calidad de los datos** es frecuentemente la dimensión más reveladora y la más incómoda. Muchas organizaciones descubren durante este diagnóstico que sus datos de negocio tienen calidad insuficiente para alimentar un sistema de IA de manera confiable: duplicados sin reconciliar en los datos maestros de clientes, valores nulos en campos críticos para el caso de uso objetivo, fechas en formatos inconsistentes que producen errores en el pipeline de procesamiento, o sesgos de muestreo que hacen que los datos históricos solo representen los casos más frecuentes y no los casos extremos donde el sistema de IA sería más valioso. El perfilado de datos con Great Expectations o con consultas SQL directas sobre las tablas candidatas — calculando métricas de completitud (% campos no nulos), unicidad (%

registros sin duplicados), y validez (% valores dentro del rango esperado) — produce una evidencia objetiva del estado de la calidad de los datos que es más útil que cualquier declaración del equipo de datos sobre su calidad.

El **diagnóstico de capacidades del equipo** evalúa el conocimiento existente en cuatro disciplinas que el AI Engineer enterprise necesita combinar: ingeniería de software (CI/CD, Docker, APIs REST, testing), ingeniería de datos (pipelines de datos, SQL, calidad de datos, orquestación), conocimiento de IA (LLMs, embeddings, RAG, evaluación de modelos), y operaciones y seguridad (cloud infrastructure, IAM, gestión de incidentes). La evaluación honesta de estas capacidades — mediante autoevaluación calibrada por el líder técnico, no solo mediante declaraciones de los miembros del equipo sobre sus habilidades — identifica las brechas que requieren contratación, capacitación, o apoyo externo durante la fase inicial del proyecto.

La **priorización de casos de uso** es el cuarto componente del diagnóstico: antes de invertir en infraestructura, el equipo debe identificar cuáles casos de uso de IA tienen el mayor impacto potencial para el negocio, los datos más disponibles y de mejor calidad para soportarlos, y la menor complejidad técnica de integración. Esta priorización produce la lista de quick wins que el Capítulo 10 desarrolla en la sección siguiente.

Componentes del diagnóstico inicial

- **Inventario de stack tecnológico:** tabla de capacidades existentes vs. requeridas por el caso de uso objetivo, con cada brecha acompañada de la estimación de tiempo para cubrirla — no de la lista de componentes que habría que comprar o construir, sino del tiempo real de implementación y configuración.
- **Evaluación de calidad de datos:** perfilado con Great Expectations o dbt tests sobre las fuentes de datos candidatas para el primer caso de uso, con métricas de completitud, unicidad, validez, y consistencia, y con identificación de las transformaciones de limpieza necesarias antes del uso.
- **Assessment de capacidades del equipo:** matrix de habilidades por competencia (Python/data engineering, cloud infrastructure, LLMs/embeddings, evaluación y testing de IA) con evaluación técnica verificable — no solo autoevaluación — que identifica los gaps más críticos para el proyecto.
- **Inventario y priorización de casos de uso:** evaluación de cada candidato por impacto estimado (medible en términos de horas ahorradas, errores reducidos, o

velocidad de proceso), disponibilidad y calidad de datos (sin datos de calidad suficiente no hay caso de uso viable), y complejidad técnica de integración.

- **Baseline de métricas actuales:** documentación del estado actual de los procesos que los casos de uso de IA buscan mejorar — tiempo actual de proceso, tasa actual de error, costo actual por unidad — para poder calcular el ROI real después del despliegue con comparación pre/post.

Buena práctica: El diagnóstico inicial debe completarse en no más de 4-6 semanas y producir un informe técnico con recomendaciones accionables — no un plan estratégico de 3 años, sino una lista de las próximas 10 acciones técnicas con responsables y estimaciones. Un diagnóstico que tarda 6 meses es una señal de que el proceso no está diseñado para producir decisiones sino para documentar el estado actual sin comprometerse con un camino de mejora.

Con el diagnóstico como punto de partida, la sección siguiente identifica los criterios para seleccionar los proyectos de quick win que construyen credibilidad interna y validan la plataforma con datos reales antes de la inversión en componentes más complejos.

Módulo 11 – Capítulo 10 – Sección 02

Quick wins técnicos: proyectos de alto impacto y bajo riesgo para construir credibilidad interna

La credibilidad interna de un equipo de AI Engineering no se construye con propuestas de transformación de largo plazo sino con evidencia de resultados funcionales en producción. Un piloto de IA que dura 18 meses y produce una demo que nunca llega a producción es el tipo de proyecto que destruye la confianza de los stakeholders de negocio en la IA como inversión viable — independientemente de la solidez técnica del trabajo. Los quick wins son el antídoto: proyectos técnicamente contenidos, con integración real a los sistemas existentes, que producen resultados medibles en producción en 6-10 semanas y que generan el capital de credibilidad que permite justificar inversiones más grandes en la plataforma.

El criterio de selección de un quick win no es la facilidad técnica — es la combinación de alto impacto de negocio medible, bajo riesgo de integración, y datos existentes de calidad suficiente. Un chatbot de soporte interno que responde preguntas sobre políticas de recursos humanos usando los documentos del portal de empleados cumple este criterio: el impacto es medible (tiempo ahorrado en preguntas al equipo de RRHH, reducible a horas por semana), el riesgo de integración es bajo (fuente de datos estática con una sincronización semanal, sin integración a sistemas críticos de negocio), y los datos existen y son accesibles (los PDFs de políticas están en SharePoint o en un drive interno). Este tipo de caso de uso puede llegar a producción en 6-8 semanas con un equipo de 2-3 personas, y produce evidencia tangible del valor de la plataforma que los stakeholders pueden verificar ellos mismos.

Los **criterios de exclusión** son igualmente importantes: un quick win no debe requerir integración con sistemas core del negocio (ERP, CRM, bases de datos de clientes) porque esas integraciones introducen dependencias organizacionales que extienden el tiempo de entrega más allá del horizonte de un quick win. No debe requerir modificación de flujos de trabajo existentes de equipos que no están directamente involucrados en el proyecto — la resistencia organizacional a cambios en los flujos de trabajo es el mayor riesgo de entrega de los proyectos de IA. Y no debe depender de datos que requieran transformación o limpieza

significativa antes de ser usables — el proceso de diagnóstico de la sección anterior debe haber identificado si los datos están en condiciones de uso.

La **secuencia de quick wins** importa más que la selección de cada proyecto individual. El primer quick win debe ser el más conservador — bajo impacto pero también bajo riesgo — para aprender el proceso de entrega de un sistema de IA en el entorno real de la organización: cómo funciona el proceso de aprobación de seguridad para nuevas herramientas, qué requiere el equipo de infraestructura para aprovisionar el entorno, qué tan lento es el proceso de revisión de contratos con proveedores de LLM. El segundo quick win puede ser más ambicioso, aprovechando el conocimiento del entorno que se acumuló en el primero. El tercer quick win puede comenzar a construir componentes reutilizables de la plataforma: el prompt registry, el sistema de evaluación, el pipeline de indexación para RAG.

Criterios para seleccionar quick wins enterprise

- **Alto impacto medible:** el impacto debe poder expresarse en métricas de negocio concretas antes de comenzar el proyecto — no "mejorará la experiencia del usuario" sino "reducirá el tiempo de respuesta de soporte de 24 horas a 2 horas para el 60% de las consultas, ahorrando X horas al equipo de soporte por semana."
- **Datos existentes y accesibles:** el caso de uso debe poder completarse con datos que ya existen, que son accesibles con las autorizaciones actuales del equipo, y que tienen calidad suficiente para el caso de uso — sin requerir un proyecto de data quality de 3 meses como prerequisite.
- **Sin integración a sistemas críticos:** el primer quick win debe usar fuentes de datos de baja criticidad (documentación interna, tickets históricos de soporte, políticas en PDF) y no requerir modificación de sistemas core del negocio.
- **Evaluable con criterios claros:** el caso de uso debe tener criterios de éxito definidos que permiten medir si el sistema está funcionando correctamente — sin criterios medibles, es imposible saber si el quick win fue un éxito o no, lo que hace imposible usar el resultado como argumento para inversiones siguientes.
- **Entregable en 6-10 semanas:** si el análisis técnico muestra que el caso de uso requiere más de 10 semanas con el equipo disponible, no es un quick win — es un proyecto mediano que requiere una propuesta formal diferente.

Nota del Arquitecto: El error más frecuente al seleccionar quick wins es confundir la facilidad técnica con el impacto medible. Un proyecto puede ser técnicamente simple y producir resultados que nadie en el negocio puede ver o medir — lo que lo hace invisible como demostración de valor. La pregunta que debe responderse antes de iniciar un quick win no es "¿podemos construir esto?" sino "¿quién en el negocio va a notar la diferencia cuando esto esté en producción, y cómo lo va a medir?"

Con los quick wins en producción como evidencia del valor de la plataforma, la sección siguiente aborda cómo construir los componentes de plataforma de manera incremental sin detener la entrega de nuevos casos de uso mientras la infraestructura madura.

Módulo 11 – Capítulo 10 – Sección 03

Construcción de plataforma incremental: comenzar simple y añadir complejidad con evidencia

La tentación más común en la construcción de plataformas de IA enterprise es diseñar la plataforma completa antes de construir nada: la arquitectura de referencia con todos los componentes, el hub-and-spoke con el IDP, el sistema multi-tenant con aislamiento de datos completo, el pipeline de evaluación continua, el cost allocation por equipo. Esta estrategia produce diagramas de arquitectura impresionantes y proyectos que tardan 18 meses en entregar un primer resultado tangible, durante los cuales los requisitos del negocio cambian, los modelos de LLM evolucionan de manera que invalidan algunas decisiones de diseño, y la credibilidad del equipo se erosiona por la ausencia de resultados visibles. La alternativa es comenzar con la infraestructura mínima que permite entregar el primer caso de uso en producción, y añadir cada componente de plataforma cuando la evidencia de uso demuestre que es necesario.

El **principio de la complejidad justificada** define el proceso: no se añade un componente de plataforma hasta que existe evidencia operacional de que la ausencia de ese componente está produciendo un problema real en producción. El sistema de gestión de prompts no se construye porque "algún día vamos a necesitar versionado de prompts" — se construye cuando el equipo ha tenido que hacer rollback de un prompt en producción y el proceso manual fue tan costoso y riesgoso que justifica la inversión en el sistema formal. El pipeline de evaluación continua no se implementa porque "las mejores prácticas recomiendan evaluación automática" — se implementa cuando el equipo ha detectado una degradación de calidad en producción que habría sido detectada antes si hubiera existido el pipeline de evaluación.

La **secuencia de construcción incremental** que produce resultados verificables en cada etapa sigue una progresión lógica dictada por las dependencias entre los componentes. La primera etapa — que puede completarse en 2-4 semanas — establece la base mínima: un repositorio Git para todos los artefactos del sistema de IA (prompts, código del pipeline, configuración del entorno), un Dockerfile que garantiza la reproducibilidad del entorno de inferencia, y un entorno de staging separado del entorno de producción donde probar los

cambios antes del despliegue. Sin esta base, ningún componente más avanzado puede construirse con confianza. La segunda etapa — semanas 4-10 — añade el pipeline de CI/CD básico, la evaluación manual con el golden dataset como gate de calidad antes de cada despliegue, y el logging básico de trazas de producción. La tercera etapa — semanas 10-20 — introduce la evaluación automática con LLM-as-a-judge, el prompt registry con API, y los dashboards de observabilidad con las métricas de calidad definidas en el Capítulo 09.

El **criterio de añadir multi-tenancy** es particularmente importante de calibrar bien: la arquitectura multi-tenant del Capítulo 04 es esencial para plataformas que sirven a múltiples equipos de negocio, pero añadirla prematuramente — cuando la plataforma sirve a un solo equipo con un solo caso de uso — introduce complejidad que frena la velocidad sin aportar valor. El indicador de que ha llegado el momento de añadir multi-tenancy es la presencia de al menos 3-4 equipos consumidores con casos de uso activos en producción: en ese punto, la ausencia de aislamiento de datos, de rate limiting por equipo, y de cost allocation produce problemas reales que requieren el componente formal.

Nota del Arquitecto: "Comenzar simple" no significa "comenzar de cualquier manera." Hay componentes que siempre deben estar presentes desde el día uno: el control de versiones del código y los prompts, el entorno de staging separado de producción, y el logging de trazas para diagnóstico. Omitir estos tres componentes para "ir más rápido" en la primera etapa produce una deuda técnica que es extraordinariamente costosa de pagar más tarde — el refactoring de un sistema sin control de versiones ni separación de entornos tiene un costo mayor que haberlos construido desde el inicio.

Secuencia de construcción incremental de la plataforma

- **Etapas 1 (semanas 1-4) — Base mínima viable:** repositorio Git con todo el código y los prompts del sistema, Dockerfile reproducible para el servicio de inferencia, entorno de staging separado de producción, pipeline básico de CI con tests unitarios. Sin estos cuatro elementos, la plataforma no puede evolucionar de manera controlada.
- **Etapas 2 (semanas 4-10) — Calidad básica:** golden dataset con 50-100 casos curados como gate de calidad, pipeline de CD que despliega automáticamente cuando el build y los tests pasan, logging de trazas con el par (input, output) de cada petición de producción, alertas básicas de errores HTTP y latencia p95.

- **Etapla 3 (semanas 10-20) — Evaluación y observabilidad:** LLM-as-a-judge ejecutándose sobre muestras del tráfico de producción, prompt registry con API de versionado y rollback, dashboards de observabilidad con las métricas de calidad del sistema, detección básica de drift sobre la distribución de inputs.
- **Etapla 4 (semanas 20-40) — Plataforma compartida:** multi-tenancy con aislamiento de datos cuando hay 3+ equipos consumidores, cost allocation por equipo, Internal Developer Portal con templates de casos de uso, detección automática de anomalías en las métricas de calidad.
- **Criterio de avance de etapa:** cada etapa se completa solo cuando el componente añadido está en producción con datos reales — no cuando el prototipo funciona en staging. El avance basado en prototipos produce plataformas que funcionan en condiciones ideales pero no en las condiciones reales de la organización.

Idea central: La complejidad de la plataforma debe crecer al mismo ritmo que la evidencia que la justifica — no más rápido, para evitar construir componentes que nadie usa, y no más lento, para evitar acumular deuda técnica que frena la velocidad operacional. El proceso de construcción incremental es el mecanismo que calibra ese ritmo.

La sección siguiente aborda el complemento necesario de la construcción incremental: cómo identificar, clasificar, y gestionar la deuda técnica que inevitablemente se acumula durante el proceso de construcción rápida de los primeros casos de uso y los primeros componentes de plataforma.

Módulo 11 – Capítulo 10 – Sección 04

Gestión de deuda técnica en proyectos de IA: identificar, clasificar y reducir gradualmente

La deuda técnica en proyectos de IA enterprise no es equivalente a la deuda técnica en proyectos de software tradicional. En software tradicional, la deuda técnica es típicamente deuda de código: acoplamiento excesivo, falta de tests, abstracciones incorrectas que hacen difícil extender el sistema. En proyectos de IA, la deuda técnica tiene dimensiones adicionales que son más difíciles de detectar y más costosas de ignorar: la deuda de evaluación (el sistema opera sin un mecanismo formal de verificación de calidad, lo que hace imposible saber si los cambios mejoran o degradan el sistema), la deuda de datos (los datos de producción están desactualizados, tienen errores de calidad no documentados, o no representan la distribución real de inputs), y la deuda de prompt (los prompts de producción han acumulado modificaciones ad-hoc sin versioning ni justificación documentada, produciendo un estado que nadie puede reproducir de manera confiable).

El primer paso de la gestión de deuda técnica en IA es el **inventario explícito**: documentar la deuda existente con una clasificación que asigna a cada ítem un tipo, un impacto en la operación actual, y una estimación del costo de resolución. Sin este inventario, la deuda es invisible — los equipos saben que el sistema tiene problemas pero no pueden priorizar qué resolver primero porque no tienen una visión completa del estado del sistema. El inventario de deuda se mantiene como un backlog técnico separado del backlog de funcionalidades: los ítems de deuda compiten por tiempo de ingeniería de manera visible, no se acumulan silenciosamente hasta que producen una crisis.

La **clasificación de la deuda técnica en IA** por tipo permite priorizar de manera más efectiva que clasificarla solo por severidad. La **deuda de evaluación** tiene siempre la máxima prioridad porque bloquea la capacidad de mejorar el sistema de manera controlada: sin evaluación formal, cada cambio al sistema — de prompt, de modelo, de datos de RAG — es un cambio a ciegas que puede degradar la calidad sin que el equipo lo detecte. La **deuda de observabilidad** tiene la segunda prioridad porque sin observabilidad el equipo no puede diagnosticar incidentes de producción de manera eficiente — el tiempo de resolución de

incidentes es alto, lo que acumula un costo de operación que aumenta con cada incidente. La **deuda de datos** tiene alta prioridad cuando la calidad de los datos está produciendo respuestas incorrectas visibles para los usuarios; si la calidad de los datos es suficiente para el caso de uso actual, puede mantenerse como deuda de baja prioridad hasta que el caso de uso lo requiera. La **deuda de código e infraestructura** — notebooks en producción, configuración hardcodeada, sin separación de entornos — tiene alta prioridad porque produce riesgo operacional: un cambio de configuración hardcodeada que afecta al entorno de producción sin pasar por staging puede producir un incidente de alta severidad.

La **estrategia de reducción gradual** define cuánto tiempo del equipo se dedica a reducir la deuda técnica en cada sprint. La regla general es reservar el 20% del tiempo del equipo para la reducción de deuda técnica en sprints normales, y el 40% en sprints dedicados específicamente a deuda técnica (que se justifican cuando el volumen de deuda acumulada está impactando visiblemente la velocidad de entrega de nuevas funcionalidades). Esta reserva debe ser no negociable ante la presión de los stakeholders de negocio por nuevas funcionalidades: la deuda técnica no resuelta tiene un costo de interés compuesto — cada sprint sin reducción de deuda hace más difícil y costoso resolver la deuda futura, porque el sistema sobre el que la deuda opera se vuelve más complejo y más difícil de cambiar.

Tipos de deuda técnica específica de proyectos de IA

- **Deuda de evaluación:** ausencia de golden dataset, sin evaluación automática en CI/CD, calidad medida solo de manera reactiva (cuando los usuarios reportan errores) — la más costosa de ignorar porque produce un sistema que opera sin mecanismo de detección de degradaciones.
- **Deuda de observabilidad:** sin logging de trazas (input/output/latencia/costo), sin dashboards de métricas de calidad, alertas solo sobre errores de infraestructura (HTTP 5xx) pero no sobre degradación de calidad — produce tiempo de diagnóstico de incidentes alto y detección tardía de problemas.
- **Deuda de prompt:** prompts modificados sin versionado formal, sin documentación de las razones de cada cambio, sin tests que validen que los cambios no producen regresiones, prompts hardcodeados en el código en lugar de en un registro — produce un estado de los prompts de producción que nadie puede reproducir o auditar de manera confiable.
- **Deuda de datos:** datos del índice RAG desactualizados (sin proceso de sincronización con la fuente), errores de calidad documentados pero no resueltos en los datos de entrenamiento o de evaluación, datos de producción que no representan la distribución

real de inputs del negocio — produce respuestas incorrectas o desactualizadas para inputs reales.

- **Deuda de infraestructura:** configuración hardcodeada en el código (API keys, URLs de endpoints), notebooks de Jupyter en producción sin pipeline de CI/CD, sin separación de entornos staging/producción, sin Dockerfiles reproducibles — produce riesgo operacional alto: cualquier cambio de configuración puede afectar a producción sin proceso de validación.

Nota del Arquitecto: La deuda técnica de evaluación es la más común y la más ignorada en proyectos de IA enterprise porque no produce fallos visibles de inmediato — el sistema sigue respondiendo, solo que nadie sabe si las respuestas son correctas. El equipo que opera un sistema de IA sin evaluación formal está en la misma situación que un piloto que vuela sin instrumentos en condiciones de visibilidad nula: puede funcionar durante un tiempo, pero el primer problema que encuentre no tendrá los datos para diagnosticarlo ni la capacidad de saber si la corrección que aplicó funcionó.

Con la gestión de deuda técnica como práctica continua, la sección siguiente articula el checklist de producción que sintetiza los criterios de madurez técnica que un sistema de IA enterprise debe cumplir antes de escalar a volúmenes de tráfico y complejidad de integración propios del entorno enterprise.

Módulo 11 – Capítulo 10 – Sección 05

Checklist de madurez técnica antes de escalar: los 6 dominios de preparación para producción enterprise

Escalar un sistema de IA que no está listo para producción enterprise es uno de los errores más costosos que puede cometer un equipo de AI Engineering: los problemas que existen a baja escala se amplifican de manera no lineal a alta escala, y resolverlos con el sistema en producción activa con tráfico real es significativamente más difícil, más riesgoso, y más costoso que resolverlos antes de escalar. El checklist de madurez técnica es la herramienta que previene este error: una verificación sistemática de los 6 dominios de preparación para producción enterprise que debe completarse antes de aumentar el tráfico, antes de incorporar nuevos equipos consumidores a la plataforma, y antes de integrar el sistema con sistemas core del negocio.

Los 6 dominios del checklist no son independientes — tienen dependencias explícitas que determinan la secuencia de verificación. El dominio de **infraestructura y despliegue** es la base: sin un pipeline de CI/CD funcional, sin entornos separados, y sin capacidad de rollback automatizado, ningún otro dominio puede verificarse de manera confiable porque cualquier problema detectado en los otros dominios no puede resolverse de manera segura. El dominio de **evaluación y calidad** es el segundo: sin evaluación formal, el sistema puede escalar pero el equipo no tendrá visibilidad sobre si la calidad se degrada bajo carga. El dominio de **seguridad y cumplimiento** es crítico para entornos regulados: escalar un sistema que procesa datos regulados sin haber completado las verificaciones de cumplimiento puede producir obligaciones legales que requieren descomisionar el sistema o aplicar cambios urgentes bajo presión.

El primer dominio — **infraestructura y despliegue** — verifica que el sistema puede desplegarse de manera reproducible y revertirse de manera segura. Los criterios concretos son: pipeline de CI/CD con ejecución automática de tests antes de cada despliegue a producción; entorno de staging que recibe el 100% de los cambios antes de producción con datos que representan la distribución de producción; Dockerfile o definición de IaC que hace el entorno completamente reproducible; proceso de rollback que puede completarse en menos

de 15 minutos sin intervención manual en el código; y proceso de canary deployment que permite dirigir el 5-10% del tráfico al nuevo deployment antes de la promoción completa.

El segundo dominio — **evaluación y calidad** — verifica que el equipo puede detectar degradaciones de calidad de manera automática. Los criterios son: golden dataset con al menos 100 casos curados que representa la distribución de inputs de producción; evaluación automática ejecutándose como gate en el CI/CD con threshold de calidad explícito; evaluación continua sobre el tráfico de producción (al menos el 5% del tráfico evaluado con LLM-as-a-judge); alertas automáticas cuando el quality score cae más del 5% respecto a la media histórica; y runbook documentado para el proceso de diagnóstico y resolución de una degradación de calidad detectada.

El tercer dominio — **seguridad y cumplimiento** — verifica que el sistema cumple los requisitos de seguridad y regulatorios del entorno enterprise. Los criterios son: revisión de seguridad aprobada por el equipo de seguridad de la organización; clasificación de los datos que el sistema procesa (si incluye datos PII o datos regulados, los controles correspondientes están implementados y documentados); logs de auditoría para todas las peticiones con retención definida por el marco regulatorio aplicable; controles de acceso (RBAC o ABAC) que garantizan que solo los usuarios autorizados pueden acceder al sistema; y proceso documentado para gestionar una solicitud de datos de un regulador o de un usuario (derecho de acceso, derecho de eliminación).

El cuarto dominio — **observabilidad** — verifica que el equipo puede detectar, diagnosticar, y resolver incidentes en producción de manera eficiente. Los criterios son: logging de trazas completas (input, output, latencia, costo de tokens) para todas las peticiones de producción con retención de al menos 30 días; dashboards con las métricas clave (latencia p50/p95/p99, error rate, quality score, costo diario) visibles en tiempo real; alertas configuradas con thresholds basados en los SLOs definidos para el sistema; runbooks actualizados para los tipos de incidentes más frecuentes; y proceso de on-call definido con responsables y escalation path para incidentes de producción fuera del horario laboral.

El quinto dominio — **gestión de costos** — verifica que el equipo tiene visibilidad y control sobre el costo del sistema antes de escalar el tráfico. Los criterios son: presupuesto mensual definido con alertas automáticas al 70% y al 90% del presupuesto; costo por tipo de petición desagregado (tokens de entrada, tokens de salida, embeddings, almacenamiento vectorial); proyección de costo bajo el volumen de tráfico post-escalado con escenario optimista, base, y pesimista; estrategias de optimización de costo implementadas en orden de impacto (prompt

caching, semantic caching, model routing según el Capítulo 07); y proceso de aprobación para cambios que incrementan el costo mensual más del 20%.

El sexto dominio — **operaciones** — verifica que el sistema puede operar en producción sin dependencia constante de los ingenieros que lo construyeron. Los criterios son: documentación de operaciones que permite a alguien sin conocimiento previo del sistema diagnosticar los problemas más frecuentes; proceso de actualización del golden dataset y de los prompts que puede completarse por el equipo de operaciones sin modificar el código; plan de contingencia para fallo del proveedor de LLM (modelo alternativo o proceso manual de fallback); proceso de comunicación de incidentes a los stakeholders y usuarios afectados; y revisión trimestral del sistema que incluye análisis de los incidentes del período y actualización del roadmap de mejoras.

Checklist consolidado de los 6 dominios

Infraestructura y Despliegue

- [] Pipeline de CI/CD con tests automáticos antes de cada despliegue a producción
- [] Entorno de staging separado de producción con datos representativos
- [] Proceso de rollback completable en < 15 minutos sin modificación de código
- [] Definición de IaC o Dockerfile que garantiza reproducibilidad del entorno
- [] Proceso de canary deployment para validar cambios con tráfico real parcial

Evaluación y Calidad

- [] Golden dataset con ≥ 100 casos curados con criterios de calidad acordados con el negocio
- [] Evaluación automática como gate en CI/CD con threshold explícito (ej. quality score ≥ 0.85)
- [] Evaluación continua sobre $\geq 5\%$ del tráfico de producción con LLM-as-a-judge
- [] Alertas automáticas ante degradación de calidad (quality score cae $> 5\%$ vs. media histórica)
- [] Runbook para diagnóstico y resolución de una degradación de calidad detectada

Seguridad y Cumplimiento

- [] Revisión de seguridad aprobada por el equipo de seguridad de la organización
- [] Clasificación de datos procesados por el sistema con controles correspondientes implementados
- [] Logs de auditoría para todas las peticiones con retención definida por el marco

regulatorio

- [] RBAC o ABAC implementado con principio de mínimo privilegio para acceso al sistema
- [] Proceso documentado para gestionar solicitudes de datos (acceso, eliminación) de usuarios o reguladores

Observabilidad

- [] Logging de trazas completas (input, output, latencia, costo) con retención ≥ 30 días
- [] Dashboards en tiempo real con métricas clave (latencia p50/p95/p99, error rate, quality score, costo)
- [] Alertas basadas en SLOs definidos para el sistema (latencia, error rate, quality score)
- [] Runbooks actualizados para los tipos de incidentes más frecuentes del sistema
- [] Proceso de on-call definido con responsables y escalation path para incidentes fuera de horario

Gestión de Costos

- [] Presupuesto mensual definido con alertas automáticas al 70% y 90% del presupuesto
- [] Costo por tipo de petición desagregado y visible en el dashboard de costos
- [] Proyección de costo bajo el volumen post-escalado (optimista/base/pesimista)
- [] Estrategias de optimización implementadas en orden de impacto (prompt caching primero)
- [] Proceso de aprobación para cambios que incrementan el costo mensual $> 20\%$

Operaciones

- [] Documentación de operaciones para diagnóstico de problemas sin conocimiento previo del sistema
- [] Proceso de actualización de golden dataset y prompts sin modificación de código
- [] Plan de contingencia para fallo del proveedor de LLM (modelo alternativo o fallback manual)
- [] Proceso de comunicación de incidentes a stakeholders y usuarios afectados documentado
- [] Revisión trimestral planificada con análisis de incidentes y actualización del roadmap

Nota del Arquitecto: Este checklist no es una lista de requisitos perfectos sino una herramienta de diagnóstico de brechas. Un sistema puede tener un 70% de los ítems completados y estar en condiciones de escalar si las brechas restantes corresponden a los dominios de menor riesgo para el caso de uso específico. La decisión de escalar con brechas existentes debe tomarse de manera explícita y documentada — con el riesgo de cada brecha

identificado, la mitigación en lugar de la solución formal aceptada, y la fecha comprometida para la resolución. Lo que nunca es aceptable es escalar sin conocer las brechas.

La sección de cierre del módulo articula la perspectiva que unifica todos los principios cubiertos: la gestión de la complejidad como el trabajo central del AI Engineer enterprise, y cómo la descomposición sistemática de esa complejidad en partes comprensibles es la competencia que distingue los sistemas que escalan de los que colapsan.

Módulo 11 – Capítulo 10 – Sección 06

Cierre del módulo: la gestión de la complejidad como competencia central del AI Engineer enterprise

El Módulo 11 comenzó con una pregunta implícita: ¿por qué los sistemas de IA que funcionan en el laboratorio o en el prototipo fallan cuando se despliegan en entornos enterprise reales? La respuesta no tiene que ver con la calidad de los modelos de lenguaje ni con la sofisticación de las técnicas de prompt engineering. Tiene que ver con la complejidad del entorno: la heterogeneidad de los sistemas legacy que no tienen APIs limpias, la diversidad de los equipos de negocio con requisitos distintos y a veces contradictorios, la rigidez de los marcos regulatorios que imponen controles que complican la arquitectura, y la dificultad de medir y mejorar la calidad de sistemas que no tienen una función de error diferenciable.

Los diez capítulos del módulo construyeron un conjunto de respuestas a esa complejidad. La **arquitectura de referencia** del Capítulo 02 descompone el sistema en capas con responsabilidades claras, lo que permite razonar sobre cada capa de manera independiente y asignar equipos con especialidades distintas a cada una. Los **patrones de integración** del Capítulo 03 descomponen la heterogeneidad de los sistemas legacy en tipos de problemas conocidos — el Adapter para la traducción de protocolos, el Anti-Corruption Layer para el aislamiento de semánticas incompatibles, el Strangler Fig para la migración incremental — con soluciones probadas para cada tipo. El **modelo multi-tenant** del Capítulo 04 descompone la diversidad de requisitos de los equipos consumidores en tres modelos de tenancy con trade-offs explícitos, lo que convierte la pregunta "¿cómo servimos a múltiples equipos?" en una decisión de arquitectura con criterios claros.

El **LLMOps** del Capítulo 05 descompone el problema de la calidad en producción en componentes separados: el dataset de evaluación como representación de los criterios de calidad del negocio, el evaluador como mecanismo de medición, el CI/CD gate como mecanismo de control, y el prompt registry como mecanismo de gestión del artefacto central del sistema. El **RAG con control de acceso** del Capítulo 06 descompone el problema del conocimiento empresarial en los componentes que lo integran: la segmentación de permisos, la recuperación filtrada por autorización, la fusión de conocimiento estructurado e inestruct

urado, y la gestión del ciclo de vida del índice. La **gestión de costos** del Capítulo 07 descompone el costo total en sus componentes — tokens, embeddings, almacenamiento, observabilidad — y ordena las estrategias de optimización en una secuencia de impacto creciente con esfuerzo creciente.

El **cumplimiento normativo** del Capítulo 08 descompone la aparente complejidad de cuatro marcos regulatorios paralelos en una matriz de controles compartidos, revelando que la mayoría de las obligaciones técnicas son las mismas bajo GDPR, HIPAA, SOC 2, y AI Act — implementadas una vez de manera correcta, no cuatro veces de manera aislada. La **medición de la madurez** del Capítulo 09 descompone el estado del sistema en cinco dimensiones medibles con criterios de evidencia verificable, convirtiendo la pregunta "¿somos maduros?" en cinco preguntas técnicas con respuestas objetivas. Y el **plan de implementación** del Capítulo 10 descompone el proceso de construcción de la plataforma en cuatro etapas secuenciales con criterios de avance claros, la gestión de la deuda técnica en tipos clasificados y priorizados, y la preparación para producción en seis dominios con criterios verificables.

La descomposición sistemática de la complejidad en partes comprensibles no es solo una estrategia técnica — es el trabajo central del AI Engineer enterprise. La complejidad no puede eliminarse de los sistemas que operan en entornos empresariales reales; solo puede gestionarse, y gestionarse bien requiere las herramientas que este módulo proporciona.

Este módulo concluye aquí; el Módulo 12 completa el ciclo con el proyecto final integrador, donde el lector aplica las técnicas y decisiones de arquitectura cubiertas en los módulos anteriores a un caso de uso enterprise real de principio a fin.

"La complejidad de los sistemas no se puede evitar — solo se puede gestionar. Y la manera de gestionarla es descomponiendo el sistema en partes que se puedan entender, construir, y verificar de manera independiente." — Fred Brooks, autor de *The Mythical Man-Month*, cuya observación sobre la complejidad esencial de los sistemas de software aplica con precisión redoblada a los sistemas de IA enterprise, donde la complejidad técnica y la complejidad organizacional se superponen en cada decisión de arquitectura.